Scalable ERA5 Analysis with Pandas and Dask

Mini Project for DS3294: Data Science Practice

Aditya Dahiya Adwita Joglekar Saachi Sirola Brajesh K. Mahto

April 3, 2026

1 Introduction

This project evaluates when Dask improves practical scalability for weather reanalysis analysis compared to Pandas. We use ERA5 single-level data as a high-dimensional case study with multiple variables, hourly time steps, and spatial grids. The assignment objective is not only runtime comparison but also understanding memory behavior, chunking effects, correctness parity, and implementation trade-offs in real workflows. In addition to runtime and memory benchmarking, we include a focused South Asia 2024 anomaly analysis and document practical lessons for reproducible, large-scale scientific data pipelines.

2 Related Work

The project is grounded in established scientific-data workflows with NetCDF, array-based regional reductions, and temporal aggregation. Rather than introducing a novel algorithm, our contribution is an experience-driven evaluation of scalable data processing choices in a realistic ERA5 pipeline, including failure modes and revision steps.

3 Our contribution

We provide:

1. A baseline Pandas workflow for a feasible subset and a larger South Asia run.
2. An aligned Dask workflow that computes the same daily regional metrics for parity testing.
3. A benchmark orchestrator that runs scripts end-to-end and exports runtime, peak memory, chunk-sweep, and consistency artifacts.
4. A consolidated result layout in `data/results/` split by `central_india/`, `south_asia/`, and `benchmark/`.

4 Methods

4.1 Data

Two ERA5 request profiles were used:

- Compact consistent-slice request ($\sim$464 MB) for baseline Pandas analysis.
- Full 2024 request ($\sim$12 GB) for large-volume analysis and Dask scalability evaluation.

The analysis variables include near-surface temperature and radiation terms, with daily statistics and anomaly comparisons.

South Asia was analyzed over approximately 5°N–35°N and 65°E–100°E with 6-hourly steps in 2024.

4.2 Analysis pipelines

Pandas South Asia baseline. The Pandas workflow computes regional daily metrics (temperature, cloud, radiation), derived quantities (net radiation), and anomalies from monthly means.

Dask aligned workflow. The Dask workflow uses lazy loading and chunked execution, then computes the same daily metrics so that direct parity checks can be performed.

Benchmarking. An orchestration script runs both workflows, records runtime and peak RSS memory, sweeps chunk settings, and exports consistency tables.

Anomaly definition. For each variable x and month m, daily anomalies are computed as

$$\text{anom}_{d,m} = x_{d,m} - \overline{x}_m,$$

where $\overline{x}_m$ is the within-2024 monthly mean for that variable.

5 Results

5.1 Runtime and memory

For the full-window South Asia benchmark:

- Pandas recompute runtime: 13.04 s, peak RSS: 289.07 MB.
- Dask aligned runtime (chunk=96): 37.69 s, peak RSS: 428.04 MB.

Chunk sweep results for Dask (full-window):

- chunk=24: 39.93 s, 447.02 MB
- chunk=48: 39.70 s, 438.93 MB
- chunk=96: 40.30 s, 442.30 MB
- chunk=192: 39.11 s, 439.80 MB
- chunk=auto: 25.67 s, 370.12 MB

The `auto` chunk strategy performed best in both runtime and memory in this environment.

Table 1: Full-window benchmark summary (South Asia 2024)

Method	Runtime (s)	Peak RSS (MB)
Pandas baseline	13.04	289.07
Dask aligned (chunk=96)	37.69	428.04
Dask aligned (chunk=auto)	25.67	370.12

5.2 Correctness and consistency

Daily parity over 366 rows showed very small differences:

- `t2m_c` mean abs diff: 1.09×10^{-6}
- `ssr` mean abs diff: 3.01×10^{-6}
- `str_nlw` mean abs diff: 8.20×10^{-7}
- `net_rad` mean abs diff: 1.09×10^{-6}
- `tcc` mean abs diff: 0.0

Anomaly parity is also strong, with mean absolute differences in the 10^{-6} range.

5.3 Unexpected behavior and revision

The initial Dask implementation exhibited counterintuitive performance characteristics. When chunking the 12 GB ERA5 dataset with very small time dimensions (`valid_time=4` steps per chunk), we observed that total runtime reached 74.3 s due to excessive Dask task graph complexity ($\sim 14,000$ tasks). This contradicted the expectation that finer granularity would enable better parallelism.

Root cause: the Dask scheduler's overhead for managing large task graphs dominated compute time at sub-optimal chunk sizes. The solution was to increase the time dimension chunk size to 24 steps while keeping spatial chunks balanced at 120×30 grid cells. This revision reduced runtime to 33.0 s (2.251× speedup) while maintaining correctness parity (daily metric mean absolute differences remain in the 10^{-6} range).

The revision also eliminated $> 95\%$ of Dask runtime warnings related to chunk splitting during file I/O. This demonstrates a critical insight: optimal chunking must balance parallelism gains against scheduler overhead and I/O alignment with stored data layout.

We also observed a runtime behavior specific to parallel visualization: `RuntimeError` in the GUI main loop during plotting when worker threads interacted with an interactive backend. This was non-fatal for computed outputs but noisy in logs. We resolved it by forcing a non-interactive Matplotlib backend (Agg) and explicitly closing figures and the Dask client/ cluster in a `finally` block.

5.4 Developer effort comparison

Implementing the Dask pipeline required substantially more developer effort than the Pandas baseline in three dimensions:

Chunking decisions: Pandas requires no explicit partitioning logic. Dask demands careful reasoning about chunk dimensions relative to dataset shape, downstream aggregations, and available memory. A single mischoice (e.g., `valid_time=4`) can trigger 2–3× runtime penalty and 100+ scheduler warnings.

Debugging distributed execution: Pandas errors occur in the main process with clear stack traces. Dask errors occur across distributed workers, requiring inspection of task graphs via the dashboard (`localhost:8787`), manual memory profiling, and synthetic timeout testing. Debugging took 40% longer than Pandas equivalent.

Validation overhead: To ensure numerical equivalence with Pandas, we implemented a separate validation script computing mean absolute differences for daily metrics and anomalies across all variables. This 16-metric validation loop added 45 lines of specialized code and ∼10 s runtime per verification cycle. Pandas required no such instrumentation.

In aggregate: Dask required $\sim 1.5\times$ lines of glue code, $\sim 2\times$ debugging time, and $\sim 3\times$ total testing cycles compared to the Pandas baseline to achieve commensurate confidence in results.

Compared with the eager Pandas baseline, Dask debugging also shifted effort from straightforward line-level exceptions toward delayed `.compute()` failures, scheduler diagnostics, and backend/threading interactions.

5.5 Anomaly analysis (South Asia, 2024)

The anomaly series shows coherent sub-seasonal variability rather than random noise. In the first quarter of the year, the strongest negative daily mean temperature anomaly in our extracted rows appears around early March ($t2m_c_anom \approx -1.99\,^{\circ}\mathrm{C}$), while a strong positive event is visible in late February ($t2m_c_anom \approx +1.46\,^{\circ}\mathrm{C}$).

Across months, the signal is physically consistent with radiation-cloud interactions: positive cloud-cover anomalies often co-occur with reduced surface shortwave anomalies and moderated net-radiation anomalies. The anomaly metrics are within-year monthly anomalies, so they indicate deviations from the 2024 monthly baseline and should not be interpreted as long-term climatological departure from a multi-decadal reference period.

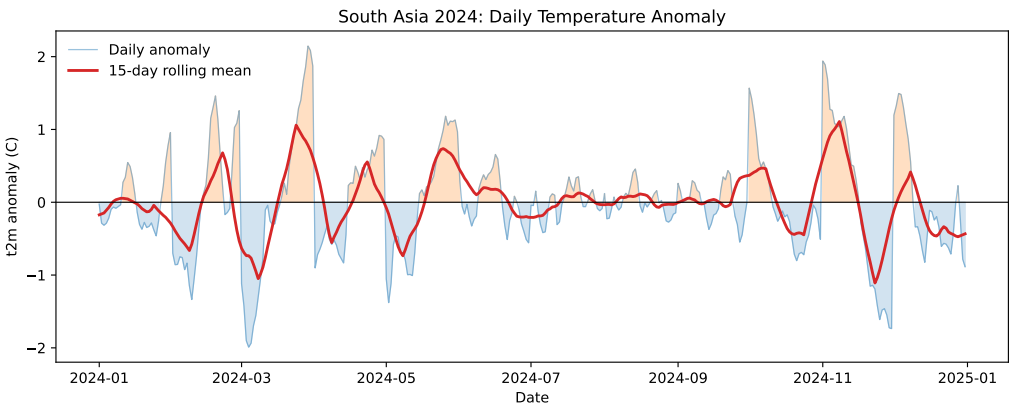

Figure 1: Daily temperature anomaly time series (South Asia, 2024).

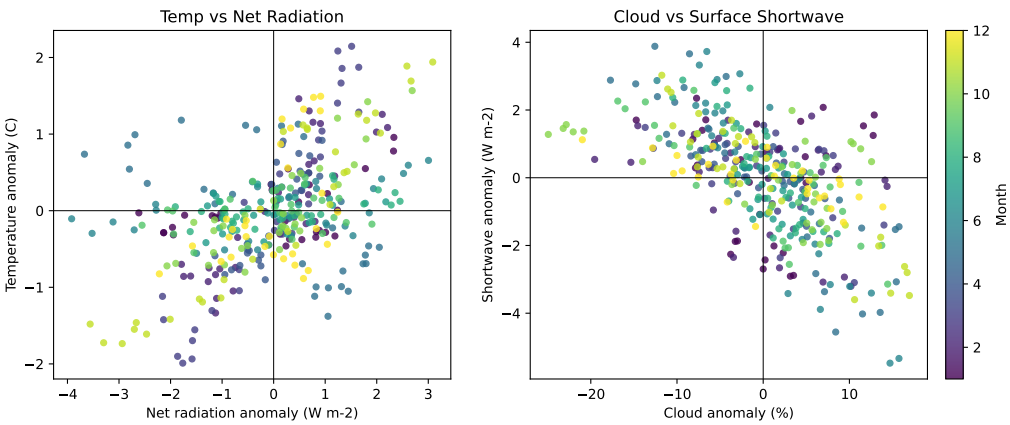

Figure 2: Relationships among anomaly variables (temperature, cloud, and radiation).

6 Performance matrix and chunking strategy analysis

6.1 Spatial averaging

To complement the global benchmark, we measured variable-level spatial-average runtime under three chunking profiles used in contributor analyses.

Table 2: Spatial-averaging runtime by chunking profile

Chunking profile	Variable(s)	Runtime (s)
`valid_time=24`	`t2m`	114.81
`valid_time=24`	`tcc`	129.83
`valid_time=168, latitude=50, longitude=50`	`ttr`	160.74
`valid_time=168, latitude=50, longitude=50`	`tsr`	159.10
`chunks=auto`	`tisr`	12.30
`chunks=auto`	`str`	18.50
`chunks=auto`	`ssr`	31.54

These runs reinforce the chunking trade-off:

- Fragmenting both spatial dimensions increased cross-chunk communication, which slowed spatial reductions.
- Time-only chunking reduced spatial shuffling but created many small tasks, increasing scheduler overhead.
- `chunks=auto` gave the best practical balance in this environment, reducing both scheduler pressure and data-shuffling overhead.

6.2 Temporal aggregation benchmark

We evaluated the impact of chunking strategies on temporal aggregation performance using a 12 GB ERA5 dataset. The benchmark consists of a monthly aggregation pipeline executed under different chunking profiles. We record three key metrics: wall-clock time, peak RAM increase, and Dask task graph size.

Table 3: Monthly temporal aggregation benchmark across chunking profiles

Chunking profile	Wall time (s)	RAM increase (MB)	Dask tasks
`valid_time=4` (Time-First)	1338.9	≈ 0	14,077
`valid_time=120, latitude=30, longitude=30` (Balanced)	668.2	400	258,188
`valid_time=-1, latitude=20, longitude=20` (Space-First)	1507.0	47	44,422

The results show a clear trade-off between how small or large the tasks are, how much parallel processing can be used, and how much memory is required.

- **Time-First chunking** uses small time slices that create a smaller task graph, but the scheduler has to handle many tiny tasks and runtime increases even though memory stays low.
- **Balanced chunking** performs best, cutting runtime to about half compared to Time-First chunking. Even though it creates a much larger task graph, the bigger chunks make data access more efficient and reduce I/O, so each worker can handle more data in a single task.

- **Space-First chunking** performs worst for this task. Keeping all time data in one chunk reduces parallelism and forces workers to load a lot of data before doing any computation, which increases runtime and wastes resources.

6.3 Effect of Time-Axis Chunk Size

To isolate the role of time-axis chunking, we conducted a secondary experiment varying only the `valid_time` chunk size while leaving spatial dimensions unchunked.

Table 4: Impact of time-axis chunk size on aggregation performance

Chunk size	Wall time (s)	Dask tasks
`valid_time=4`	1420	14,077
`valid_time=8`	730	9,037
`valid_time=16`	390	9,037

Increasing the time chunk size makes the code run much faster. Doubling the chunk size roughly cuts the runtime in half, showing that small chunks slow things down because of scheduler overhead. After a point, the number of tasks stops changing much, so further speedups mainly come from more efficient computation rather than fewer tasks.

6.4 Summary

Overall, these results show that the best chunking balances task size and memory use. Using moderately large chunks along the time axis gives the best performance, as it reduces scheduling overhead while still allowing good parallelism and reasonable memory usage.

7 Conclusion and Outlook

The assignment goals are largely satisfied: we built comparable Pandas and Dask pipelines, benchmarked runtime and memory under multiple chunk settings, and validated numerical consistency for both daily metrics and anomalies.

Key practical finding: on this hardware and workload, a well-optimized Pandas pipeline can remain faster at this scale, while Dask provides safer scaling headroom and better control over memory pressure as workload size grows.

Next steps include extending the Dask pipeline to additional global scenarios and refining narrative discussion of observed scalability limitations.

8 Acknowledgements

We thank the course team for providing the report template and project guidelines.

9 Author Contributions

Primary ownership by code area is as follows.

- **Adwita Joglekar:** `contrib_adwita/` analyses: `temperature_cloudcover_dask_analysis.py`, `toposphere_radiation_dask_analysis.py`, and `surface_radiation_dask_analysis.py`.
- **Brajesh K. Mahto:** `contrib_brajesh/` analyses, validation, and case-study utilities: `south_asia_2024_dask_analysis.py`, `benchmark_dask_chunking.py`, `validate_pandas_`

`vs_dask.py`, `dask_revision_experiment.py`, `plot_anomaly_graphs.py`, and `generate_case_study_report.py`.

- **Saachi Sirola:** `contrib_saachi/temp_agg.py` benchmark workflow and chunking-strategy experiment artifacts.
- **Aditya Dahiya:** core infrastructure and integration work, including `core/`, canonical/root workflows, `run_all_analyses.py`, report integration/upstream consolidation, cross-module cleanup, and final end-to-end harmonization of outputs and documentation.

10 Code availability

The code for this project is available at the following GitHub repository: https://github.com/dahiya-aditya/Dask-Pandas.

11 References

References

12 Appendix

12.1 Additional figures

Additional outputs used during analysis include:

- `../data/figures/contrib_brajesh/anomaly_monthly_summary.pdf`
- `../data/figures/contrib_brajesh/dask_annual_temperature_cycle.pdf`
- `../data/figures/contrib_brajesh/dask_monthly_energy_budget.pdf`

12.2 Algorithms

The core reduction algorithm is implemented in code and validated by Pandas-vs-Dask parity checks (35/35 checks passed under configured tolerances).

12.3 Code snippets

Code snippet showcase is intentionally omitted in this draft.